COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	October 21, 2026	**DATE SUBMITTED:**	October 21, 2026
NAME:	Quiming, Kadaphy James A. Larracas, Jerry Matthew L.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14.1 ALGORITHM COMPLEXITY

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Algorithm 1	**O(1)**	**Unbound or target value not found within specified range**	**O(1) : Finds the target value at first try** **Error : The target value is outside the boundary/search range**
Algorithm 2			

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Algorithm 1 (Bubble Sort)	Algorithm 2 (Insertion Sort)
Best Case	**100 :** 14 ms	**100 :** 0 ms
	1000 : 1296 ms	**1000 :** 4 ms
	10000 : 129385 ms	**10000 :** 43 ms
Worst Case	**100 :** 47 ms	**100 :** 16 ms
	1000 : 4660 ms	**1000 :** 1521 ms
	10000 : 496908 ms	**10000 :** 159329 ms

```
Best Case (100) :
Bubble Sort: 14 microseconds
Insertion Sort: 0 microseconds

Worst Case (100) :
Bubble Sort: 47 microseconds
Insertion Sort: 16 microseconds
```

```
Best Case (1000) :
Bubble Sort: 1296 microseconds
Insertion Sort: 4 microseconds

Worst Case (1000) :
Bubble Sort: 4660 microseconds
Insertion Sort: 1521 microseconds
```

```
Best Case (10000) :
Bubble Sort: 129385 microseconds
Insertion Sort: 43 microseconds

Worst Case (10000) :
Bubble Sort: 496908 microseconds
Insertion Sort: 159329 microseconds
```



```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <ctime>
#include <random>
#include <algorithm>
#include <thread>
#include <functional>

void bubbleSort(int array[], size_t size);
void insertionSort(int array[], size_t size);
void runBubbleSort(int array[], int size, long long &result);
void runInsertionSort(int array[], int size, long long &result);

int main() {
    int sizes[] = {100, 1000, 10000};

    for (int size : sizes) {
        std::vector<int> bestCase(size);
        std::vector<int> worstCase(size);

        for (int i = 0; i < size; i++) {
            bestCase[i] = i + 1;
            worstCase[i] = size - i;
        }

        for (int test = 0; test < 2; test++) {
            std::vector<int> bubbleArr =
                (test == 0) ? bestCase : worstCase;

            std::vector<int> insertionArr = bubbleArr;

            long long bubbleTime = 0, insertionTime = 0;

            std::thread th_bubble(
                runBubbleSort, bubbleArr.data(), size,
                std::ref(bubbleTime));

            std::thread th_insertion(
                runInsertionSort, insertionArr.data(), size,
                std::ref(insertionTime));

            th_bubble.join();
            th_insertion.join();

            if (test == 0)
                std::cout << "\n\nBest Case (" << size << ") : ";
            else
                std::cout << "\n\nWorst Case (" << size << ") : ";

            std::cout << "\nBubble Sort: " << bubbleTime << " microseconds";
            std::cout << "\nInsertion Sort: " << insertionTime << " microseconds";
        }
    }

    return 0;
}

void bubbleSort(int array[], size_t size) {
    for (int i = 0; i < size; i++) {
        for (int j = i + 1; j < size; j++) {
            if (array[i] > array[j]) {
                std::swap(array[i], array[j]);
            }
        }
    }
}

void insertionSort(int array[], size_t size) {
    for (int i = 1; i < size; i++) {
        int temp = array[i];
        int j = i - 1;

        while (j >= 0 && array[j] > temp) {
            array[j + 1] = array[j];
            j--;
        }

        array[j + 1] = temp;
    }
}

void runBubbleSort(int array[], int size, long long &result) {
    auto start = std::chrono::steady_clock::now();

    bubbleSort(array, size);

    auto end = std::chrono::steady_clock::now();

    result = std::chrono::duration_cast<std::chrono::microseconds>(
        end - start).count();
}

void runInsertionSort(int array[], int size, long long &result) {
    auto start = std::chrono::steady_clock::now();

    insertionSort(array, size);

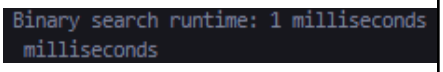
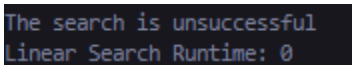
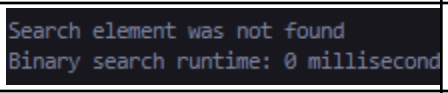
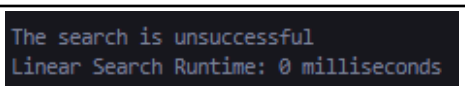
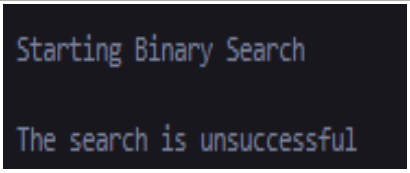
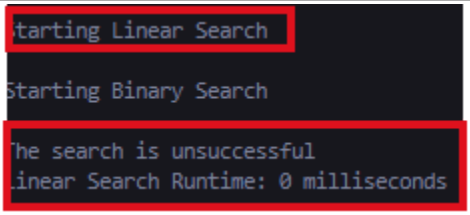
    auto end = std::chrono::steady_clock::now();

    result = std::chrono::duration_cast<std::chrono::microseconds>(
        end - start).count();
}
```

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILOA: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.

Input Size (N)	Elapsed Time for Binary Search (Worst Case)	Elapsed Time for Linear Search (Worst Case)
1,000		
10,000		
100,000		
Larracas(Analysis)	All I can observe is that both searches are not successful, I don't know if our code works or there is something wrong to it, but Binary search does run a bit longer than linear search.	
Quiming(Analysis)	Both searches were not successful with the inputs. Binary search ran a bit longer than linear search, in some tests.	

CONCLUSION:	
Larracas	This activity helped me learn about the time complexities of different sorting and searching algorithms based on how fast the device runs. In the supplementary, we weren't sure if we did the code right or the experimental tools right but I know that when running the input size 100,000 or more, it can make your vs code crash or close all by itself.
Quiming	In this activity, I learned about comparing the time complexities of two algorithms. I don't think that I haven't totally mastered space and time complexities.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Debugging**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We just needed help with debugging but it didn't replace our original idea iat all.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.